



Universidade Federal de São Paulo
Instituto de Ciência e Tecnologia
Introdução às Redes Neurais Artificiais - Turma I

Desenvolvimento de um modelo de rede neural convolucional para o diagnóstico de casos de pneumonia infantil através de imagens de radiografias torácicas.

Discente: Gabriel Delgado Panovich de Barros
RA 176313

Docente responsável: Prof. Dr. Marcos Gonçalves Quiles

São José dos Campos
2025/2

Resumo

A implementação de algoritmos para o apoio à decisão clínica, como diagnóstico por imagem, é um desafio fundamentado na complexidade de definição de métodos de confiabilidade para as saídas fornecidas por esses programas. O uso dessas ferramentas é essencial para automatizar processos onerosos, agilizando o acesso do paciente a tratamentos essenciais e, situacionalmente, urgentes. Este projeto visa fornecer uma abordagem para esse gargalo através de arquiteturas de Redes Neurais Convolucionais (CNNs), aplicadas à detecção de pneumonia em radiografias torácicas. Ao comparar modelos próprios com modelos de referência (estado da arte), utilizando técnicas de *Data Augmentation* para potencializar a generalização do modelo, busca-se atingir uma arquitetura que equilibre custo e precisão, de modo que as aplicações do modelo sejam viáveis no diagnóstico por imagem. Os resultados demonstraram que arquiteturas simplificadas, compostas por apenas dois blocos convolucionais, superaram abordagens mais complexas em estabilidade e generalização, atingindo acurácia de $\sim 94\%$ e F1-Score de ~ 0.96 , validando a eficácia de modelos enxutos para a identificação de opacidades pulmonares.

Conteúdo

1	Introdução	3
2	Referencial teórico	3
2.1	Redes Neurais Artificiais	4
2.2	Redes Convolucionais	4
2.3	AlexNet	4
2.4	Visual Geometry Group (VGG)	5
2.5	YOLO	5
3	Metodologia	5
3.1	Conjunto de dados (Dataset)	5
3.2	Pré-processamento e Data Augmentation	6
3.3	Arquitetura da Rede Neural	7
3.4	Configuração do treinamento	8
3.5	Métricas de avaliação	9
4	Resultados e Discussão	10
4.1	Modelo Baseline	10
4.2	Modelo Proposto	12
4.3	Comparativo e Conclusão dos Experimentos	15
5	Conclusão	15
5.1	Trabalhos Futuros	15
A	Código Principal de Treinamento	19
B	Classe de Modelo de Rede Neural	20

1 Introdução

A introdução de novas tecnologias no campo da saúde é uma prática que traz incontáveis benefícios em diversos escopos. O desenvolvimento da radiologia, por exemplo, possibilitou maior rapidez na identificação, diagnóstico e tratamento de uma variedade de doenças e condições médicas[1, 2].

No contexto do diagnóstico por imagem, no entanto, persiste o gargalo da avaliação por especialistas, uma etapa historicamente difícil de ser automatizada com precisão. A abordagem consistia em sistemáticas etapas que demandam tempo, mesmo quando utilizando redes neurais rasas[3]. No caso em que essas redes eram utilizadas, havia também o custo computacional e a requisição de inúmeros especialistas para a definição desses classificadores[4, 5, 6].

Pelo seu poder de processamento autônomo de imagens, as Redes Neurais Convolucionais (CNNs) tornaram-se uma alternativa disruptiva para esse cenário, como visto em [7, 8, 9, 10, 11]. Diversas arquiteturas foram desenvolvidas para adaptar-se a diferentes cenários, desde classificadores robustos como VGG[12] e AlexNet[13] até detectores em tempo real como o YOLO[9].

Assim, este projeto visa propor uma arquitetura que equilibre generalização e custo para o diagnóstico de patologias. Para validar essa abordagem, o modelo desenvolvido tem enfoque na classificação de casos de pneumonia utilizando radiografias torácicas do conjunto de dados público disponibilizado por [14].

2 Referencial teórico

Nesta seção, são apresentados os fundamentos teóricos que embasam o desenvolvimento do modelo proposto. Inicialmente, abordam-se os conceitos de Redes Neurais Artificiais (RNAs) e o surgimento do Aprendizado Profundo (Deep Learning). Em seguida, detalha-se o funcionamento das Redes Neurais Convolucionais (CNNs), arquiteturas projetadas especificamente para o processamento de dados em grid, como imagens. Por fim, discute-se a evolução histórica dessas arquiteturas através de modelos clássicos como AlexNet e VGG, bem como abordagens modernas de detecção de objetos, exemplificadas pelo algoritmo YOLO, contextualizando suas contribuições para o estado da arte no diagnóstico por imagem.

2.1 Redes Neurais Artificiais

As Redes Neurais Artificiais são modelos computacionais inspirados na estrutura biológica do cérebro humano, compostas por unidades de processamento elementares (neurônios) interconectadas. Em sua forma clássica, conhecida como Multilayer Perceptron (MLP), essas redes são capazes de aprender funções não-lineares complexas através do algoritmo de Backpropagation, que ajusta os pesos das conexões para minimizar o erro de predição.

No entanto, para o processamento de imagens, as MLPs tradicionais apresentam limitações severas. Como exigem que a imagem seja convertida em um vetor unidimensional (flattening) antes da entrada, elas descartam a estrutura espacial dos pixels (a relação de vizinhança entre eles) e sofrem com a explosão combinatória de parâmetros, tornando o treinamento inviável para imagens de alta resolução.

2.2 Redes Convolucionais

As Redes Neurais Convolucionais (CNNs) surgiram como uma solução para a limitação das MLPs em lidar com imagens, as quais exigem quantidades exorbitantes de parâmetros nesse tipo de rede neural. As CNNs preservam a estrutura espacial da imagem através de operações de convolução[15], onde filtros (ou kernels) deslizam sobre a entrada gerando mapas de características (feature maps)[16], resultados de campos receptivos locais nos neurônios ocultos[17]. A vantagem dessa abordagem é a capacidade de aplicar alterações posicionais na imagem de entrada e garantir que o modelo vai extrair características semelhantes, possibilitada pela camada de pooling.

A camada de pooling, componente crucial das CNNs, é responsável por reduzir a dimensão dos mapas de características, conferindo essa propriedade de invariância à translação.

2.3 AlexNet

A arquitetura AlexNet[18], proposta por [19], marcou o início da era moderna do Deep Learning ao vencer a competição ImageNet de 2012 com uma margem significativa de acurácia. Suas principais contribuições incluíram o uso de arquiteturas profundas (8 camadas), a introdução da função de ativação ReLU (que acelerou a convergência do treinamento ao mitigar o problema do desaparecimento de gradiente) e o uso extensivo de GPUs para processamento paralelo. Além disso, a AlexNet popularizou técnicas de regularização como o Dropout, essencial para evitar overfitting em redes grandes [13].

2.4 Visual Geometry Group (VGG)

A arquitetura VGG, desenvolvida por pesquisadores da Universidade de Oxford, consolidou a ideia de que a profundidade é um fator determinante para a performance. Sua principal inovação foi a substituição de filtros grandes (como 11×11 na AlexNet) por uma sequência de filtros pequenos de tamanho 3×3 . Essa abordagem, baseada em blocos convolucionais repetitivos, permitiu a construção de redes muito mais profundas (como a VGG-16 e VGG-19) com menos parâmetros relativos e maior capacidade de não-linearidade. O conceito de empilhar blocos de convolução seguidos de pooling, central na VGG, serve de inspiração direta para a arquitetura Baseline desenvolvida neste trabalho[12].

2.5 YOLO

Enquanto AlexNet e VGG focam na classificação de imagens (dizer o que está na imagem), o algoritmo YOLO revolucionou a tarefa de detecção de objetos (dizer o que e onde). Proposto por [9], o YOLO trata a detecção como um problema único de regressão, processando a imagem inteira de uma só vez ("You Only Look Once") através de uma única rede neural. Isso o torna extremamente rápido e apto para aplicações em tempo real, diferindo das abordagens de CNNs clássicas que processam janelas deslizantes ou propostas de região separadamente.

3 Metodologia

Para este projeto, distintos módulos foram desenvolvidos, garantindo maior legibilidade do código e concisão. A divisão em células separadas permite facilidade na visualização do fluxo geral do pipeline principal, o qual será mais detalhadamente explicado nesta seção. Por questões de concisão, o código completo deste módulo encontra-se detalhado no Apêndice A.

3.1 Conjunto de dados (Dataset)

Os dados foram reunidos em [14], na plataforma *Kaggle*. As radiografias, conforme descreve o autor, foram extraídas de um estudo de coorte retrospectivo de pacientes do Centro Médico de Mulheres e Crianças de Guangzhou. Essas imagens correspondem a exames de rotina dos pacientes.

Ressalta-se que se trata de um conjunto de dados com um leve tratamento prévio. Imagens de qualidade muito baixa ou ilegíveis foram removidas do conjunto, que poderiam ser problemáticas para o treinamento de um modelo.

Os conjuntos de treino, validação e teste foram divididos em diretórios do dataset, apesar de, nesta análise, optou-se por agrupar por conta própria os conjuntos. Isso foi necessário devido ao forte desbalanceamento da quantidade de imagens entre os conjuntos separados pelo autor do dataset: o conjunto de avaliação possuía apenas 16 imagens, 8 de cada classe. A classificação das imagens foi feita por dois especialistas, garantindo precisão no diagnóstico final[8].

3.2 Pré-processamento e Data Augmentation

O pipeline de processamento de dados foi implementado utilizando as classes primitivas Dataset e DataLoader do framework PyTorch. Essas ferramentas gerenciam o carregamento eficiente das imagens em memória e a formação dos lotes (batches) para o treinamento.

Uma etapa fundamental foi a padronização da entrada. Como as radiografias variam em dimensões e, nativamente, podem possuir três canais de cor (RGB) dependendo do formato do arquivo, todas as imagens foram redimensionadas para 224×224 pixels e convertidas explicitamente para escala de cinza (1 canal), reduzindo a complexidade computacional da primeira camada convolucional.

As transformações nas imagens do conjunto de treinamento são um ponto de destaque, pois há um nuance responsável por gerar respostas inconsistentes do modelo. As técnicas de *Data Augmentation* do PyTorch não consideram correções de borda da imagem, deixando tarjas pretas nos cantos das imagens dos dataloaders que podem agir como ruído no treinamento da rede neural. Apesar de, no caso de imagens de raios-X, as próprias imagens já apresentam cantos escuros, esse ruído poderia enviesar o modelo. Como forma de garantir a maior precisão possível, esse problema foi mitigado ao se aplicar um pequeno zoom às imagens do conjunto de treino que sofreram essas mudanças (Listing 1).

```
1 train_transforms = transforms.Compose([
2     transforms.Resize((224, 224)),
3     transforms.Grayscale(num_output_channels=1),
4     transforms.RandomAffine(
5         degrees=10,           # Rotacao leve
6         translate=(0.05, 0.05),
7         scale=(1.05, 1.15), # Zoom para evitar bordas pretas
8         fill=0
9     ),
10    transforms.ToTensor(),
11 ])
```

Listing 1: Definição das transformações do conjunto de treino

Em contrapartida, para os conjuntos de validação e teste, não foram aplicadas transformações geométricas ou de ruído. Esses conjuntos sofreram apenas o redimensionamento e a normalização de valores (transformação em tensores no intervalo $[0,1]$), garantindo que a avaliação de desempenho refletisse a capacidade do modelo em lidar com imagens originais e não alteradas.

3.3 Arquitetura da Rede Neural

Uma solução proposta em [20] avalia um modelo ResNet para a modelagem do problema. Dada a complexidade do modelo e custo elevado para a aplicação da ResNet, buscou-se encontrar uma arquitetura mais simples que, pelo menos, chegasse a resultados semelhantes aos encontrados nessa implementação.

Com base nisso, uma proposta foi iniciar os experimentos em um modelo minimalista, buscando chegar a um estado de *underfitting*, e aumentando a complexidade do modelo conforme observado espaço de melhora. Dessa forma, uma classe de modelo versátil seria essencial para permitir a flexibilidade exigida pela abordagem escolhida:

- ***SimpleCNN_Base***: Define a estrutura da rede neural. A classe (Listing 3) foi projetada para ser modular, aceitando como parâmetros a lista de filtros, o uso de regularizadores (*Dropout*, *Batch Normalization*) e o tipo de aglutinação final (*Flatten* ou *Global Average Pooling*).
- ***cnnModel***: Um encapsulamento ("wrapper") responsável por gerenciar o ciclo de vida do treinamento. Esta classe abstrai os loops de treinamento, validação e teste, além de calcular e armazenar automaticamente as métricas de desempenho (Loss, Acurácia, F1-Score)

Nota-se, na arquitetura base, o uso padronizado de kernels 3×3 com padding para manutenção da dimensionalidade espacial, seguidos pela função de ativação ReLU e Max Pooling 2×2 , uma estrutura inspirada nos blocos da VGGNet[12], priorizando filtros pequenos (3×3) para capturar detalhes finos das opacidades pulmonares. A fim de manter a concisão, o código referente a essa seção localiza-se no Apêndice B.

Para a validação experimental, duas configurações de arquitetura foram instanciadas a partir desta classe base:

1. **Modelo Baseline (Simples)**: Composto por apenas dois blocos convolucionais (32 e 64 filtros). Esta configuração não utiliza regularização (*Dropout* ou *Batch Norm*) e emprega o método tradicional de *Flatten* na camada densa. O objetivo deste modelo é estabelecer uma linha de

base (modelo minimalista) de desempenho com o menor custo computacional possível.

2. **Modelo Proposto (Completo):** Uma evolução do baseline, composta por três blocos (32, 64 e 128 filtros). Para combater a instabilidade gerada pelo aumento da profundidade, foram integradas técnicas de *Batch Normalization* após cada convolução, *Dropout* de 0.5 na camada densa e *Global Average Pooling (GAP)* para redução de parâmetros na transição para o classificador.

3.4 Configuração do treinamento

Os experimentos foram realizados utilizando a linguagem Python sobre o framework PyTorch. O processamento foi acelerado via hardware utilizando uma GPU NVIDIA GeForce RTX 3050 (4GB VRAM) e a plataforma CUDA.

Para a otimização dos pesos da rede, utilizou-se o algoritmo Adam (Adaptive Moment Estimation)[13]. A escolha deste otimizador justifica-se pela sua capacidade de ajustar taxas de aprendizado individuais para cada parâmetro, proporcionando convergência mais rápida e estável em comparação ao SGD (Stochastic Gradient Descent) clássico, especialmente em superfícies de erro não convexas.

A função de custo (Loss Function) adotada foi a Cross-Entropy Loss (Entropia Cruzada), padrão ouro para problemas de classificação, que penaliza logaritmicamente a divergência entre a probabilidade predita pelo modelo e o rótulo real da classe.

A Tabela 1 resume os hiperparâmetros fixados para o treinamento de ambos os modelos (Baseline e Completo):

Tabela 1: Hiperparâmetros utilizados no treinamento

Hiperparâmetro	Valor
Número de Épocas	30
Batch Size (Tamanho do Lote)	128
Learning Rate (Taxa de Aprendizado)	$1 \cdot 10^{-3}$ (0.001)
Otimizador	Adam
Função de Custo	Cross-Entropy
Critério de Parada/Seleção	Menor erro de validação

É importante notar a estratégia de persistência do modelo (Checkpointing). Ao final de cada época, o erro no conjunto de validação (Validation Loss) foi monitorado. O modelo final salvo em disco não corresponde necessariamente ao da última época (30^a), mas sim ao estado dos pesos na época em

que se obteve o menor erro de validação. Essa abordagem previne a seleção de um modelo que tenha entrado em regime de overfitting nos estágios finais do treinamento.

3.5 Métricas de avaliação

Para mensurar a performance dos modelos propostos, utilizou-se a análise da Matriz de Confusão, considerando os termos: Verdadeiros Positivos (VP), Verdadeiros Negativos (VN), Falsos Positivos (FP) e Falsos Negativos (FN). A partir desses valores, quatro métricas fundamentais foram calculadas:

- **Acurácia (Accuracy):** Representa a taxa global de acertos do modelo, ou seja, a razão entre todas as predições corretas e o total de amostras.

$$Accuracy = \frac{VP + VN}{VP + VN + FP + FN} \quad (1)$$

- **Precisão (Precision):** Indica a confiabilidade das predições positivas. Dentre todos os casos classificados como Pneumonia, calcula-se quantos realmente eram Pneumonia.

$$Precision = \frac{VP}{VP + FP} \quad (2)$$

- **Sensibilidade (Recall):** Também conhecida como Revocação, é uma métrica crítica em diagnósticos médicos. Ela mede a capacidade do modelo de detectar corretamente os casos positivos, minimizando os Falsos Negativos (pacientes doentes diagnosticados erroneamente como saudáveis).

$$Recall = \frac{VP}{VP + FN} \quad (3)$$

- **F1-Score:** É a média harmônica entre a Precisão e o Recall. Esta métrica é essencial para avaliar o equilíbrio do modelo, sendo especialmente útil em cenários onde há desbalanceamento entre as classes ou quando se deseja penalizar tanto falsos positivos quanto falsos negativos.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (4)$$

4 Resultados e Discussão

A análise comparativa dos experimentos revelou uma tendência notável a favor da arquitetura minimalista. Contrapondo a hipótese inicial de que redes mais profundas seriam mandatórias para a extração de características complexas em radiografias, o Modelo Baseline apresentou maior estabilidade e capacidade de generalização do que o Modelo Proposto (mais profundo e regularizado).

4.1 Modelo Baseline

O comportamento do modelo simples (2 camadas, sem regularização) pode ser observado na Figura 1. As curvas de erro de treino (training loss) e validação (validation loss) apresentam um declínio acentuado já nas primeiras épocas, indicando uma rápida convergência.

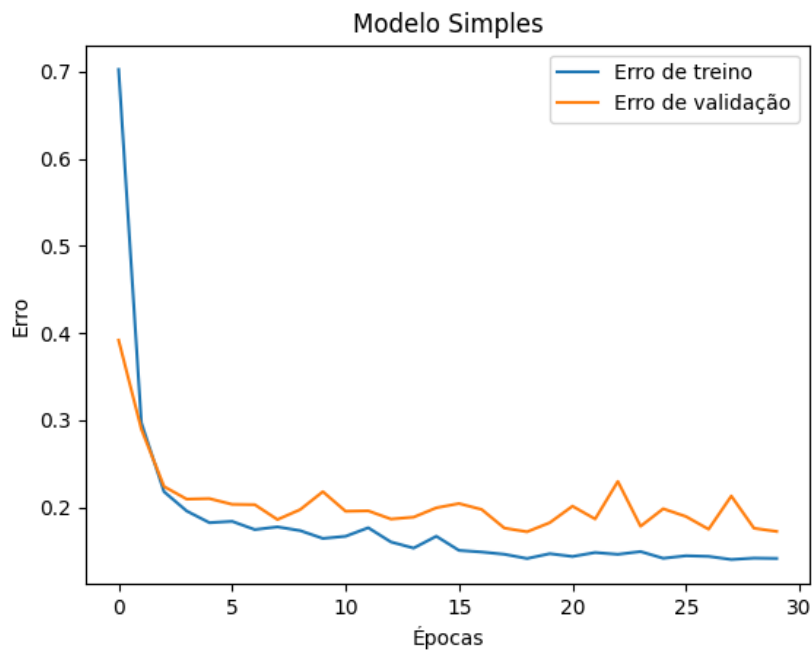


Figura 1: Curvas de aprendizado do Modelo Baseline. Nota-se a proximidade entre treino e validação.

Contrariando a ideia inicial, o modelo não apresentou *underfitting*. Ele foi capaz de aprender o problema de forma eficaz, evidente pela proximidade entre as curvas do gráfico (Figura 1), e atingir um erro de validação próximo de 0.

Os resultados finais no conjunto de teste, detalhados abaixo, corroboram a eficácia desta abordagem leve:

- **Acurácia:** $\sim 94\%$
- **Recall (Sensibilidade):** $\sim 95\%$
- **F1-Score:** $\sim 96\%$

Essa análise é reforçada pela saída do teste do modelo, evidente na Figura 2, em que é possível verificar um baixo índice de Falsos Negativos, o principal alvo de análise para um caso clínico.

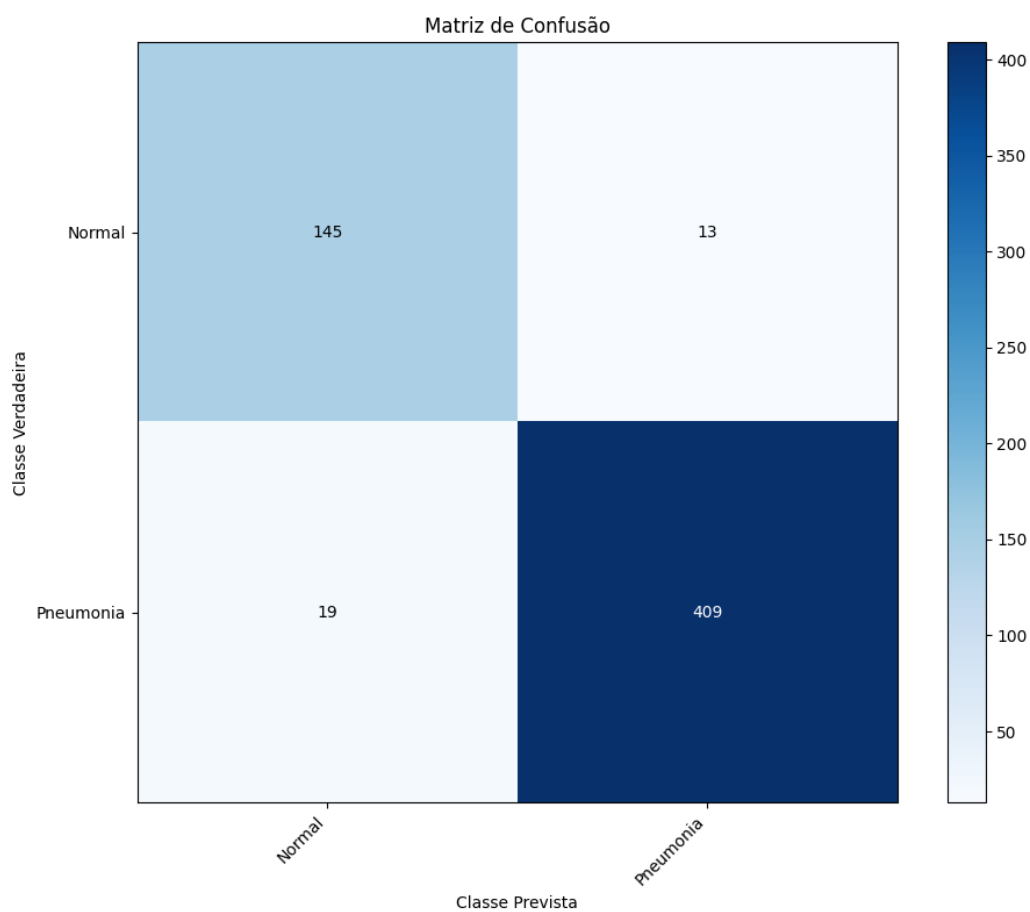


Figura 2: Matriz de Confusão do Modelo Baseline no conjunto de teste.

4.2 Modelo Proposto

Conforme os resultados do modelo anterior, poderia-se supor que o modelo mais complexo (3 camadas e regularização) apresentasse *overfitting*. Contudo, observou-se instabilidade e alta variância de generalização no modelo, como destacado pela Figura 3.

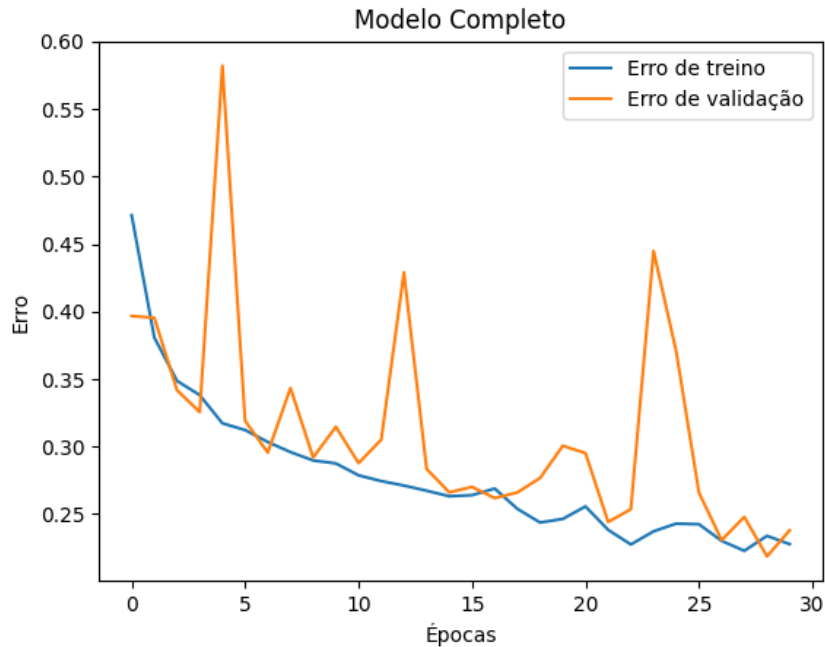


Figura 3: Curvas de aprendizado do Modelo Baseline. Nota-se a proximidade entre treino e validação.

Embora o erro de treinamento apresente um decaimento consistente, indicando que a rede é capaz de aprender as características do conjunto de treino, o erro de validação exibe alta variância e instabilidade severa.

Os picos abruptos de erro (notadamente nas épocas 4, 12 e 23) sugerem que a complexidade adicional da arquitetura dificultou a convergência do otimizador para um mínimo global estável. Diferente do overfitting clássico, onde a curva de validação diverge de forma constante para cima, aqui observa-se uma incapacidade de generalização consistente: o modelo alterna entre bons e maus desempenhos a cada época, tornando-o pouco confiável para implantação clínica, onde a previsibilidade é mandatória.

Observa-se o impacto desse problema nos resultados finais dessa abordagem:

- **Acurácia:** $\sim 90\%$
- **Recall (Sensibilidade):** $\sim 92\%$
- **F1-Score:** $\sim 93\%$

Apesar do gargalo verificado, o modelo ainda atingiu resultados significativos. Contudo, quando comparado ao modelo anterior, verifica-se uma clara inferioridade na capacidade de generalização, o que pode ser visto na matriz de confusão da Figura 4

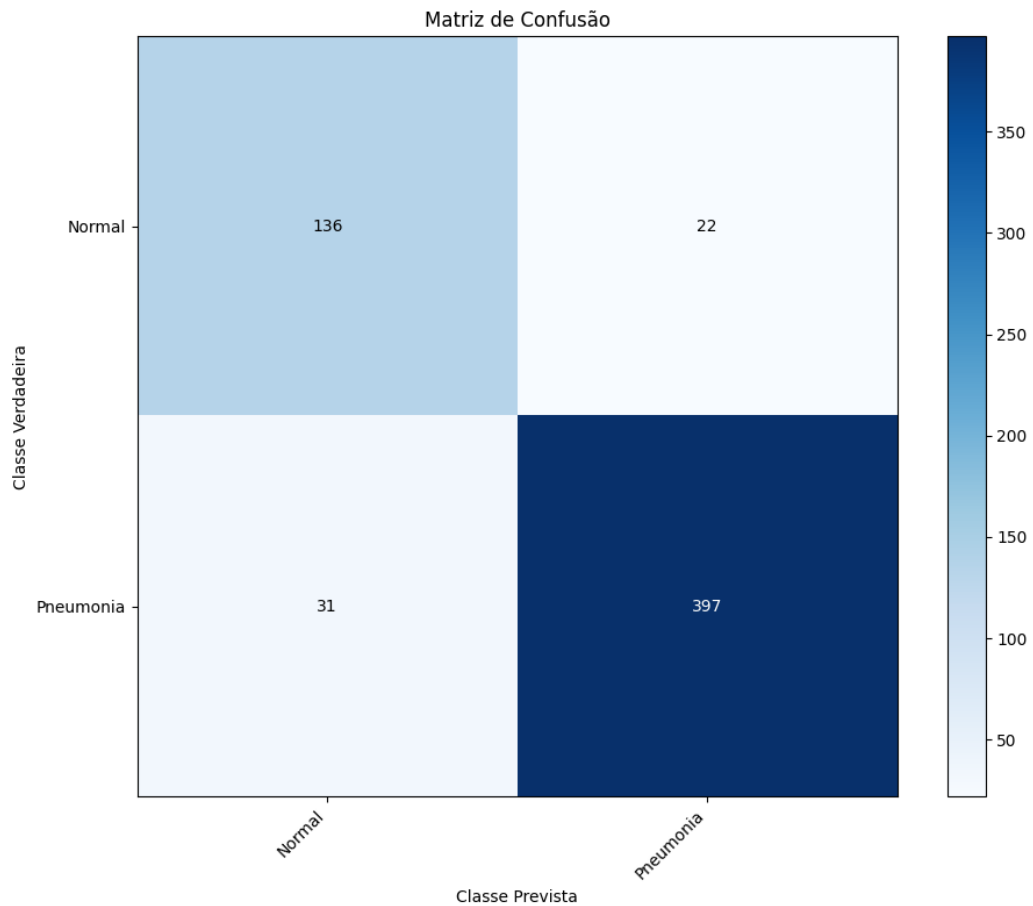


Figura 4: Matriz de Confusão do Modelo Proposto no conjunto de teste.

Diferentemente do Modelo Baseline, este modelo apresentou 31 Falsos Positivos. Ainda que um número baixo, por volta de 5% dos casos, dada a complexidade dessa abordagem, a adoção deste modelo não faz sentido quando diante do modelo minimalista.

4.3 Comparativo e Conclusão dos Experimentos

A Tabela 2 sumariza o desempenho das duas abordagens.

Tabela 2: Comparativo de desempenho entre as arquiteturas

Modelo	Acurácia	Recall	F1-Score
Baseline (Simples)	94.75%	95.56%	96.24%
Proposto (Complexo)	90.96%	92.76%	93.74%

Esses resultados permitem inferir que, para a tarefa de detecção de pneumonia neste dataset específico, a arquitetura rasa é mais eficiente. O fenômeno pode ser explicado pela natureza das características visuais da pneumonia (opacidades difusas), que podem ser capturadas satisfatoriamente nas primeiras camadas convolucionais. O aumento da profundidade, neste cenário de dados limitados (aprox. 5.000 imagens), introduziu ruído e complexidade desnecessária, validando o princípio da parcimônia no design de redes neurais médicas.

5 Conclusão

Buscou-se, através deste projeto, construir uma implementação prática de modelos de Redes Neurais Convolucionais, investigando suas aplicações na área do diagnóstico por imagem. A proposta consistia em equilibrar custo computacional com precisão clínica, valendo-se de distintas arquiteturas para avaliar os dois atributos.

Os resultados apontaram para uma direção contrária à suposta inicialmente, em que o modelo mais complexo descreveria o problema com maior precisão. Observou-se que um modelo minimalista foi capaz de modelar o problema objetivamente melhor, possivelmente resultado do volume e natureza dos dados utilizados.

Conclui-se, portanto, que a complexidade do modelo deve ser calibrada cuidadosamente em relação à disponibilidade de dados. Para datasets médicos de porte médio, o princípio da parcimônia mostrou-se valioso: arquiteturas rasas (Shallow Networks) podem oferecer generalização superior e menor custo computacional do que redes profundas treinadas do zero (from scratch).

5.1 Trabalhos Futuros

Como desdobramentos desta pesquisa, sugerem-se as seguintes direções:

- Transfer Learning: Investigar o uso de redes pré-treinadas (como ResNet[10, 20] ou DenseNet) em grandes bancos de imagens genéricas (ImageNet) para verificar se a transferência de conhecimento mitiga a instabilidade observada em arquiteturas profundas.
- Explicabilidade (XAI): Implementar algoritmos de visualização[9], como o Grad-CAM, para gerar mapas de calor sobre as radiografias. Isso permitiria validar se a rede está focando nas regiões pulmonares corretas ou se está sendo enviesada por artefatos da imagem, aumentando a confiança médica no sistema[11].
- Validação Cruzada: Aplicar K-Fold Cross-Validation para garantir que as métricas de desempenho sejam estatisticamente consistentes em diferentes partições dos dados, seguindo as práticas de seleção de modelos e validação robusta descritas em [12].

Referências

- [1] Clínica Tirol, “Radiologia: conheça essa especialidade e sua importância na medicina.” Blog da Clínica Tirol, jul 2023. Disponível em: <https://clinicatirol.com.br/2023/07/radiologia-conheca-essa-especialidade-e-sua-importancia-na-medicina/>. Acessado em: 14-12-2025.
- [2] Universidade Tiradentes, “Radiologia e a importância do diagnóstico por imagem.” Blog Unit, oct 2023. Disponível em: <https://www.unit.br/blog/radiologia-e-a-importancia-do-diagnostico-por-imagem>. Acessado em: 14-12-2025.
- [3] M. Goldbaum, S. Moezzi, A. Taylor, S. Chatterjee, J. Boyd, E. Hunter, and R. Jain, “Automated diagnosis and image understanding with object extraction, object classification, and inferencing in retinal images,” in *Proceedings of 3rd IEEE International Conference on Image Processing*, vol. 3, pp. 695–698 vol.3, 1996.
- [4] A. Hoover, V. Kouznetsova, and M. Goldbaum, “Locating blood vessels in retinal images by piecewise threshold probing of a matched filter response,” *IEEE Transactions on Medical Imaging*, vol. 19, no. 3, pp. 203–210, 2000.
- [5] A. Hoover and M. Goldbaum, “Locating the optic nerve in a retinal image using the fuzzy convergence of the blood vessels,” *IEEE Transactions on Medical Imaging*, vol. 22, no. 8, pp. 951–958, 2003.

- [6] S. Chaudhuri, S. Chatterjee, N. Katz, M. Nelson, and M. Goldbaum, “Detection of blood vessels in retinal images using two-dimensional matched filters,” *IEEE Transactions on Medical Imaging*, vol. 8, no. 3, pp. 263–269, 1989.
- [7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Commun. ACM*, vol. 60, p. 84–90, May 2017.
- [8] D. S. Kermany, M. Goldbaum, W. Cai, C. C. Valentim, H. Liang, S. L. Baxter, A. McKeown, G. Yang, X. Wu, F. Yan, *et al.*, “Identifying medical diagnoses and treatable diseases by image-based deep learning,” *Cell*, vol. 172, no. 5, pp. 1122–1131, 2018.
- [9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” *arXiv preprint arXiv:1506.02640*, 2015.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *arXiv preprint arXiv:1512.03385*, 2015.
- [11] C. Daube, T. Xu, J. Zhan, O. G. B. Garrod, and P. G. Schyns, “Grounding deep neural network predictions of human categorization behavior in understandable functional features: The case of face identity,” *Patterns*, vol. 2, no. 11, p. 100348, 2021.
- [12] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Redes Usando Blocos (VGG)*, ch. 7.2. Cambridge University Press, 2023. Versão em Português. Disponível em: https://pt.d2l.ai/chapter_convolutional-modern/vgg.html. Acessado em: 14-12-2025.
- [13] A. Pujara, “Concept of alexnet: Convolutional neural network.” Analytics Vidhya (Medium), nov 2020. Acessado: 14-12-2025.
- [14] P. Mooney, “Chest x-ray images (pneumonia).” Kaggle, 2018. Disponível em: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>. Acessado em: 14-12-2025.
- [15] 3Blue1Brown, “But what is a convolution?.” Youtube, nov 2022. Disponível em: <https://www.youtube.com/watch?v=KuXjwB4LzSA>. Acessado em: 13-12-2025.
- [16] J. Purtell, “Understanding how convolutional layers work.” Data Science Stack Exchange, 2020. Acessado: 14-12-2025.

- [17] Data Science Academy, “Campos receptivos locais em redes neurais convolucionais.” Deep Learning Book, 2019. Acessado em: 14-12-2025.
- [18] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Redes Neurais Convolucionais Profundas (AlexNet)*, ch. 7.1. Cambridge University Press, 2023. Versão em Português. Disponível em: https://pt.d2l.ai/chapter_convolutional-modern/alexnet.html. Acessado em: 14-12-2025.
- [19] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, Curran Associates, Inc., 2012.
- [20] T. Yang, “Pneumonia detection (resnets + pytorch).” Kaggle, 2020. Disponível em: <https://www.kaggle.com/code/teyang/pneumonia-detection-resnets-pytorch>. Acessado em: 15-12-2025.

A Código Principal de Treinamento

Abaixo apresenta-se a implementação completa do pipeline principal (*main*), responsável pela orquestração do carregamento de dados, treinamento e avaliação dos modelos.

```
1 # Definicoes iniciais
2 EPOCHS = 30
3 DEVICE = "cuda"
4 PATH = "db/"
5
6 os.makedirs("models", exist_ok=True)
7 os.makedirs("assets", exist_ok=True)
8 train_loader, val_loader, test_loader = getDataLoaders(PATH,
9     batch_size=128)
10 print("Exemplo de lote")
11 sampleBatch(train_loader)
12
13 simpleModel = cnnModel(
14     device=DEVICE,
15     filters_list=[32, 64] # exemplo de filtros
16 )
17
18 optimizer_simple = optim.Adam(simpleModel.model.parameters(),
19     lr=0.001)
20 loss_fn = nn.CrossEntropyLoss()
21 num_epochs = EPOCHS
22
23 # Loop de treinamento
24 best_simple_val_loss = float('inf')
25 for epoch in range(num_epochs):
26     simpleModel.trainModel(train_loader, loss_fn,
27         optimizer_simple)
28     simpleModel.validateModel(val_loader, loss_fn)
29
30     if simpleModel.val_loss[-1] < best_simple_val_loss:
31         best_simple_val_loss = simpleModel.val_loss[-1]
32         simpleModel.saveCheckpoint("models/
33             best_simple_cnn_model.pth")
34
35 # Visualizacao dos erros
36 plotLosses(
37     "Modelo Simples",
38     simpleModel.train_loss,
39     simpleModel.val_loss,
40     "Erro de treino",
41     "Erro de validacao",
42     EPOCHS,
```

```

40     "assets/Simple_CNN_Losses.png"
41 )
42
43 # Teste do modelo
44 simpleModel.model.load_state_dict(torch.load("models/
    best_simple_cnn_model.pth"))
45 simpleModel.testModel(test_loader)
46 print(f"Relatorio Final ")
47 print(f"Acuracia (Accuracy):    {simpleModel.acc*100:.2f}%")
48 print(f"Precisao (Precision):   {simpleModel.prec*100:.2f}%")
49 print(f"Sensibilidade (Recall): {simpleModel.rec*100:.2f}%")
50 print(f"F1 Score:               {simpleModel.f1*100:.2f}%")
51
52 plotCMatrix(
53     simpleModel.test_class,
54     simpleModel.predicted_class,
55     ['Normal', 'Pneumonia'],
56     "assets/Simple_CNN_ConfusionMatrix.png"
57 )

```

Listing 2: Main

B Classe de Modelo de Rede Neural

Abaixo encontra-se o código utilizado para definir a classe flexível do modelo de rede neural.

```

1 class SimpleCNN_Base(nn.Module):
2     def __init__(
3         self,
4         filters_list: List[int],
5         num_classes: int = 2,
6         dropout: bool = False,
7         batch_norm: bool = False,
8         GAP: bool = False,
9         input_size: int = 224
10    ):
11        """
12        Args:
13            filters_list: Lista que contem as dimensoes (
14                quantidade total de neuronios)
15                de cada camada de filtro da CNN.
16            num_classes: Quantidade de classificacoes
17                possiveis.
18            dropout: Parametro que determina se o modelo
19                utilizara dropout.
20            batch_norm: Define se o modelo tera batch
21                normalization.

```

```

18         GAP: Informa se o modelo utilizara Global Average
19         Pooling
20         input_size: Tamanho da imagem, utilizado para
21         quando o modelo nao conter GAP
22         """
23         super(SimpleCNN_Base, self).__init__()
24         layers = []
25         in_channels = 1 # como trabalhamos com grayscale, ha
26         apenas 1 canal
27
28         for filtro in filters_list:
29             layers.append(nn.Conv2d(in_channels, filtro,
30             kernel_size=3, padding=1))
31             if batch_norm:
32                 layers.append(nn.BatchNorm2d(filtro))
33             layers.append(nn.ReLU()) # escolhido segundo o
34             modelo da AlexNet
35             layers.append(nn.MaxPool2d(kernel_size=2, stride
36             =2))
37             in_channels = filtro
38
39         self.conv_layers = nn.Sequential(*layers)
40         self.use_gap = GAP
41         self.use_dropout = dropout
42
43         if self.use_gap:
44             self.global_pool = nn.AdaptiveAvgPool2d(1)
45             input_features = filters_list[-1]
46         else:
47             num_pools = len(filters_list)
48             final_img_size = input_size // (2 ** num_pools)
49             input_features = filters_list[-1] *
50             final_img_size * final_img_size
51
52         if self.use_dropout:
53             self.dropout = nn.Dropout(0.5)
54             self.classifier = nn.Linear(input_features,
55             num_classes)
56
57         def forward(self, x):
58             x = self.conv_layers(x)
59
60             if self.use_gap:
61                 x = self.global_pool(x)
62                 x = x.view(x.size(0), -1) # Remove as dimensoes 1
63                 x1 extras
64             else:
65                 x = x.view(x.size(0), -1) # Flatten tradicional

```

```
58         if self.use_dropout:
59             x = self.dropout(x)
60         x = self.classifier(x)
61         return x
```

Listing 3: Classe de modelo base